

Online Pharmacy Platform

High-Level Design

Architecture & System Overview

This document describes the high-level architecture of the Online Pharmacy Platform, covering system context, microservices topology, infrastructure components, frontend structure, data persistence, and cross-cutting concerns.

1. System Context

The Online Pharmacy Platform serves two actor types — Customers (browser/mobile) and Admins (dashboard). All requests flow through a React SPA to an API Gateway, which routes them to dedicated microservices. An SMTP-based email provider handles outbound notifications.

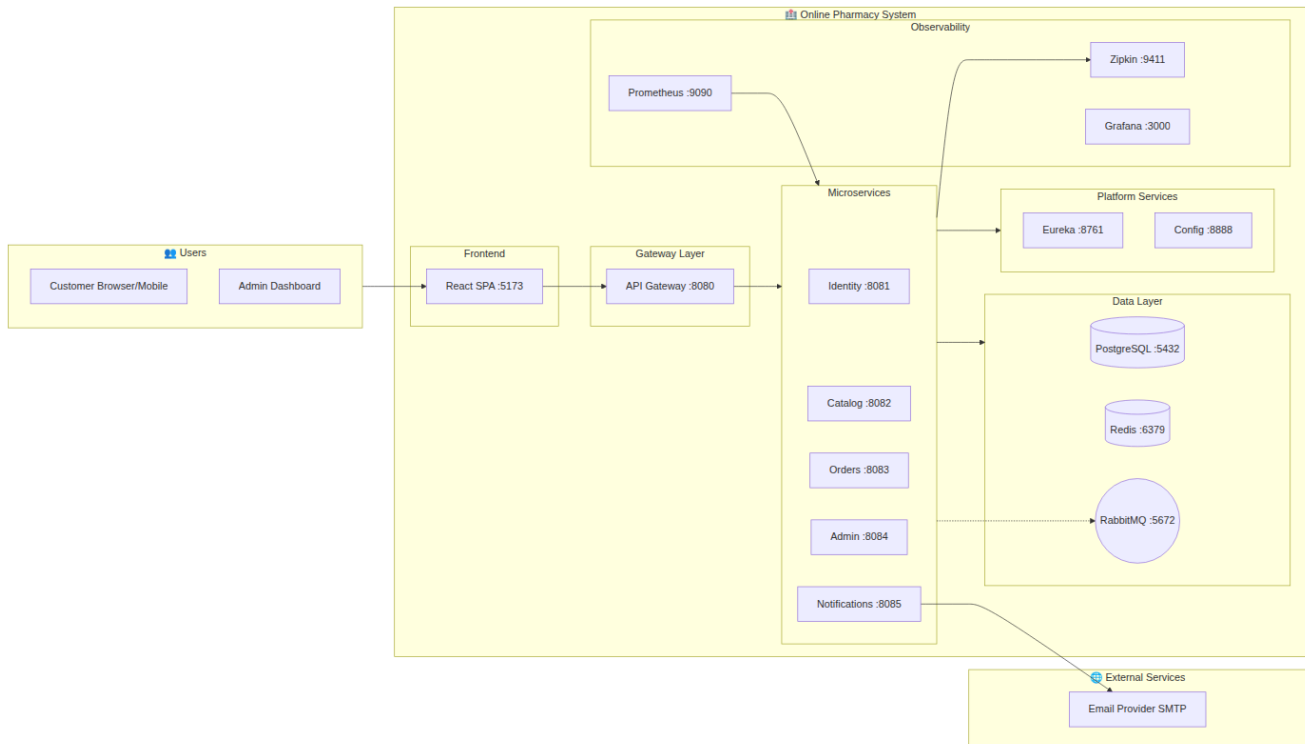


Figure 1 — System Context Diagram

2. Microservices Architecture

The backend is composed of five domain microservices, each owning its own PostgreSQL database (database-per-service pattern), orchestrated within a Docker infrastructure. Spring Cloud Gateway handles JWT validation, rate limiting, and routing. Service discovery is provided by Eureka, centralised configuration by a Config Server.

2.1 Service Inventory

Identity Service :8081 — Authentication, OTP, address management, JWT blacklisting via Redis.

Catalog Service :8082 — Medicine catalogue, categories, inventory management, prescription uploads.

Orders Service :8083 — Cart lifecycle, order processing, payment handling.

Admin Service :8084 — Dashboard KPIs, order management, prescription review — uses Feign clients.

Notifications Service :8085 — Event-driven email and in-app notifications via RabbitMQ consumers.

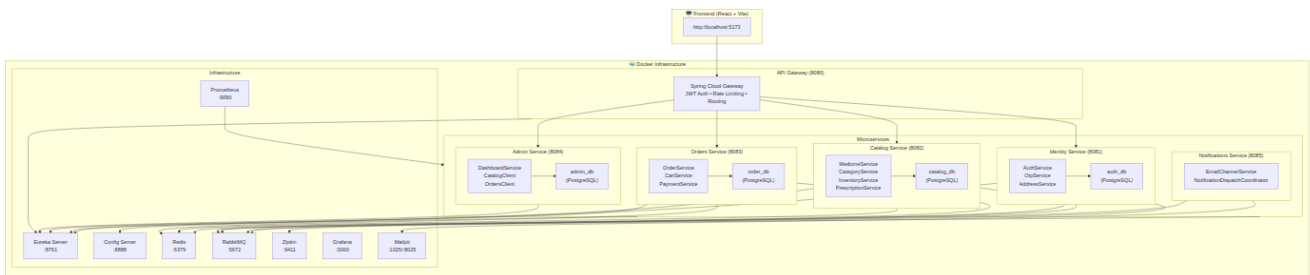


Figure 2 — Microservices & Infrastructure Diagram

3. Infrastructure Components

PostgreSQL :5432 — Relational store; each service has its own logical database (auth_db, catalog_db, order_db).

Redis :6379 — JWT blacklist, OTP caching, distributed session data for Identity and Catalog services.

RabbitMQ :5672 — Async event bus for domain events (order placed, prescription approved, payment confirmed, etc.).

Eureka :8761 — Service registry; all microservices register at startup for dynamic discovery.

Config Server :8888 — Centralised configuration for all services; backed by a Git repository.

Zipkin :9411 — Distributed tracing — end-to-end request correlation across service boundaries.

Prometheus :9090 / Grafana :3000 — Metrics scraping and dashboarding for operational visibility.

Mailpit :1025/8025 — Local SMTP sink / web UI used in development to inspect outgoing emails.

4. Frontend Architecture

The frontend is a React + Vite SPA served at port 5173. It is structured around two layouts (user-facing and admin), backed by Zustand stores for state management and a typed API layer for communication with the gateway.

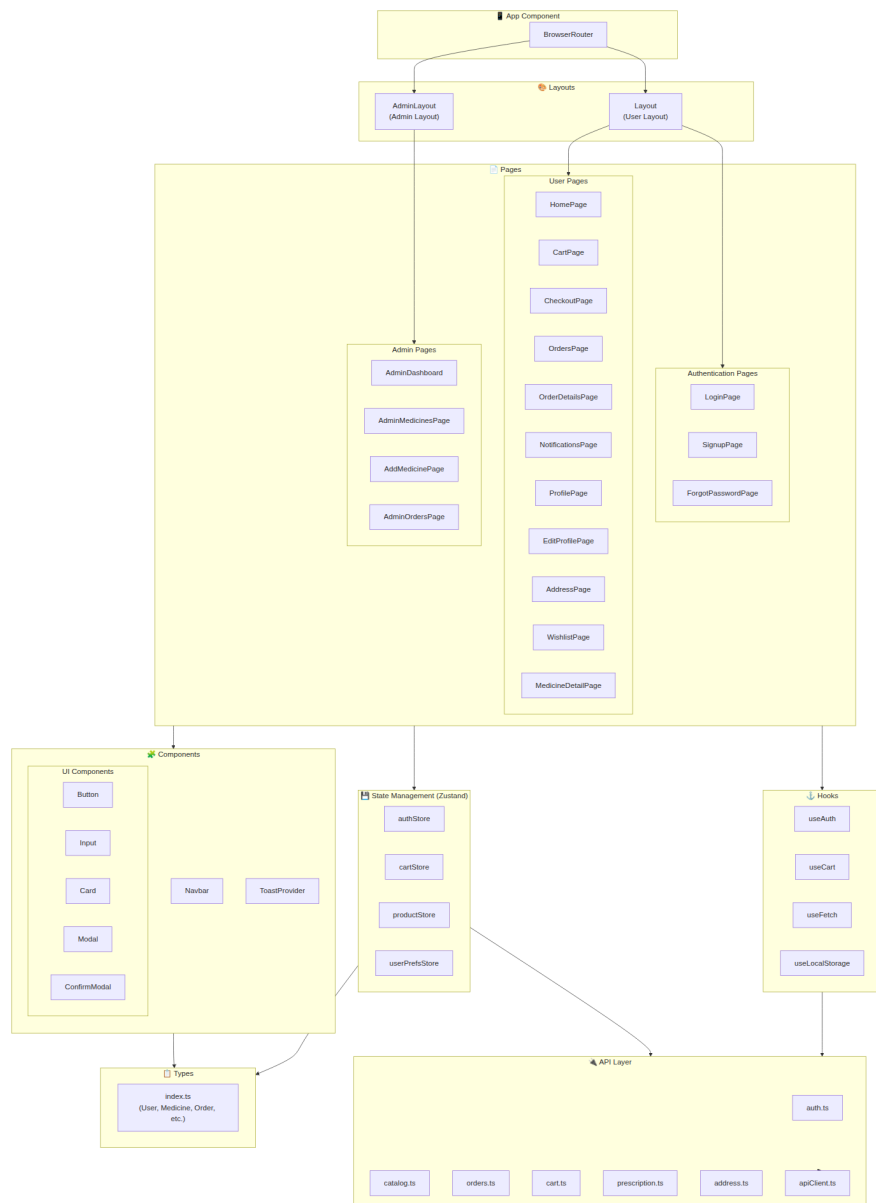


Figure 3 — Frontend Component Hierarchy

5. Shared Domain Model

5.1 Common Enumerations & Response Types

All services share a common module that defines canonical enumerations (OrderStatus, PaymentMethod, PrescriptionStatus, etc.) and generic API response wrappers (ApiResponse, ApiPaginatedResponse). A GlobalExceptionHandler provides consistent error responses across all services.

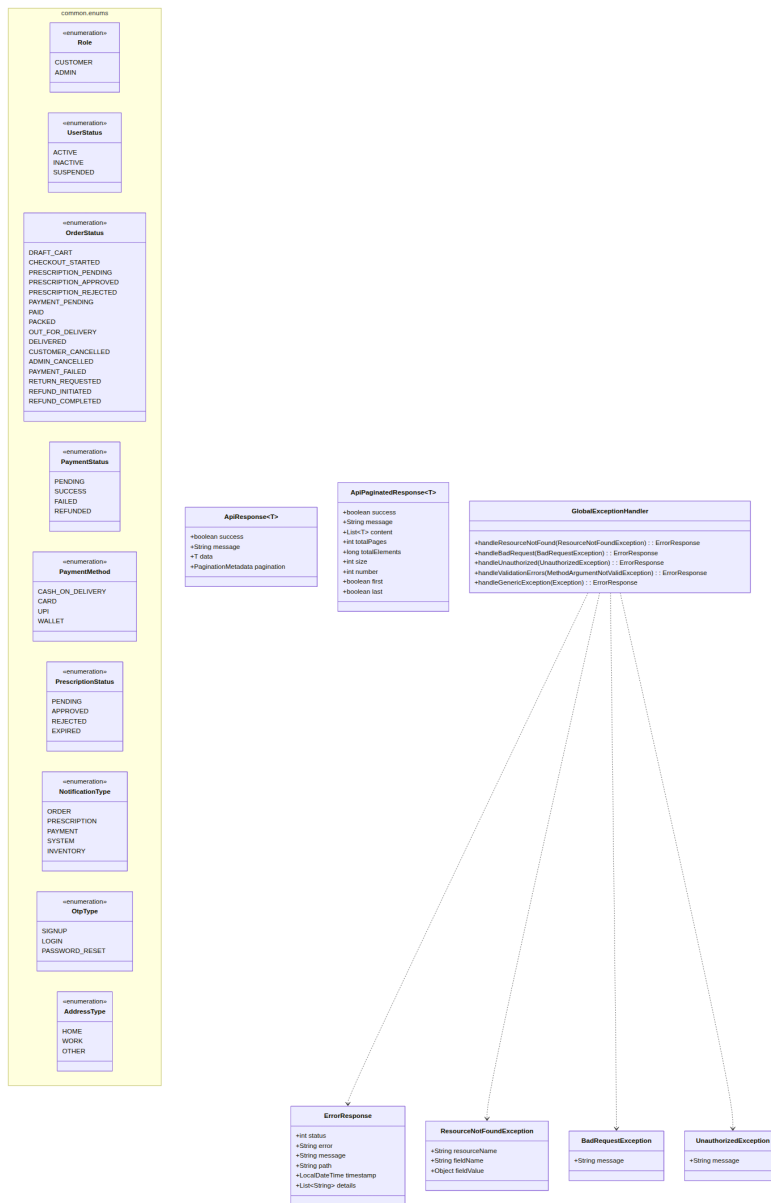


Figure 4 — Common Types & Enumerations

5.2 Domain Events

Domain events extend a common BaseEvent (eventId, correlationId, timestamp, sourceService). Events are published to RabbitMQ and consumed by the Notifications Service and other interested parties.

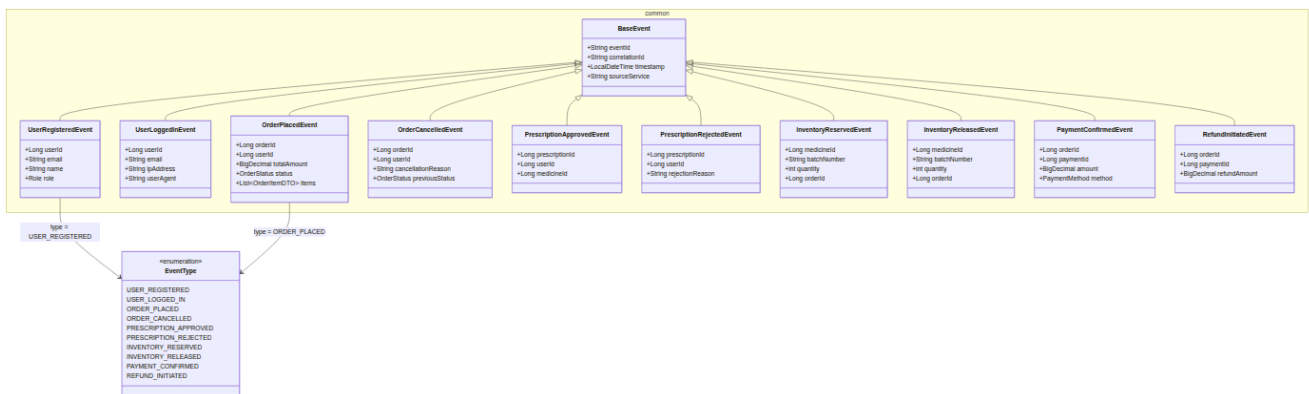


Figure 5 — Common Domain Events

6. Database Schema

Three logical databases back the platform. `auth_db` holds users, addresses, notifications, OTP records, and the JWT blacklist. `catalog_db` manages medicines, categories, inventory batches, and prescriptions. `order_db` covers carts, cart items, orders, order items, and payments.

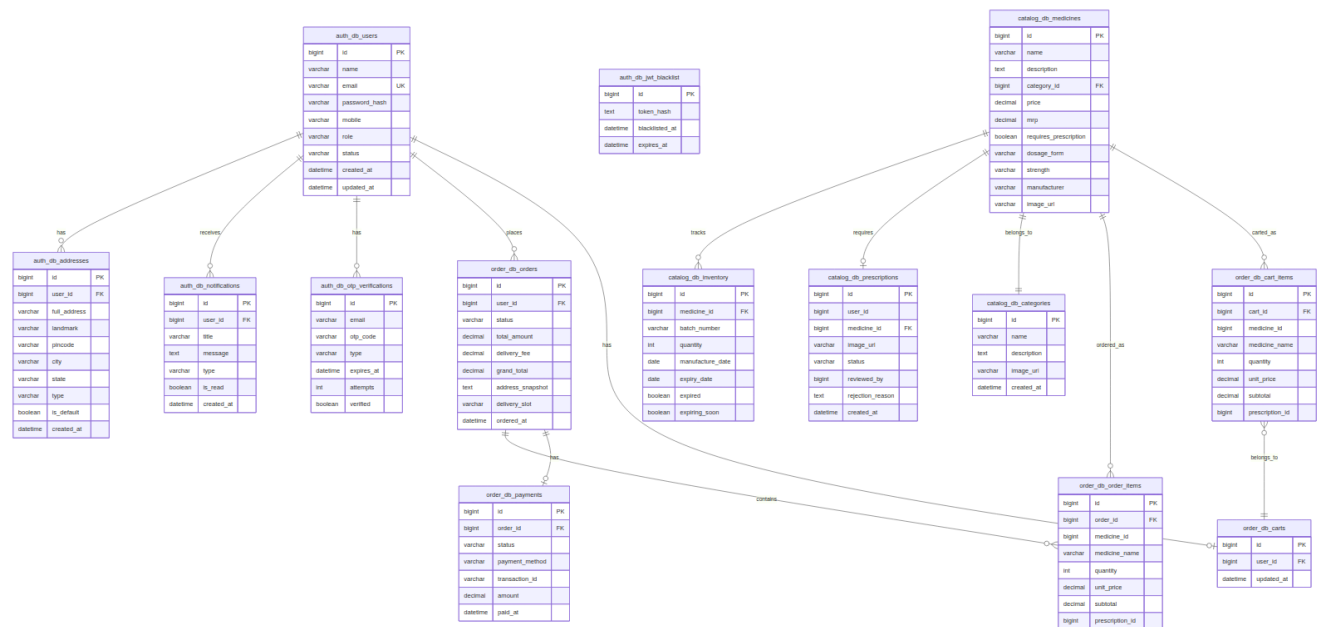


Figure 6 — Entity-Relationship Diagram

7. Key Sequence Flows

7.1 Authentication Flow

Supports simple password login, OTP-based MFA, full signup with OTP verification, and logout with Redis-backed JWT blacklisting.

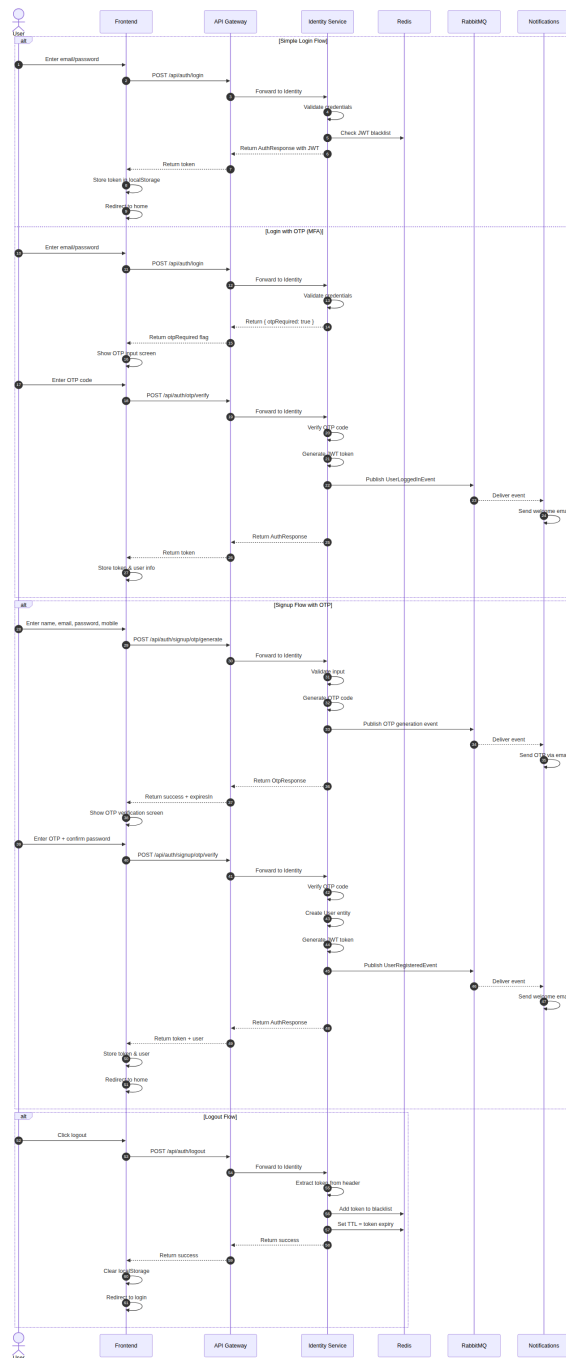


Figure 7 — Authentication Sequence Diagram

7.2 Checkout Flow

Covers add-to-cart (with and without prescription), checkout with inventory reservation, payment processing (success and failure paths), and order cancellation with inventory release.

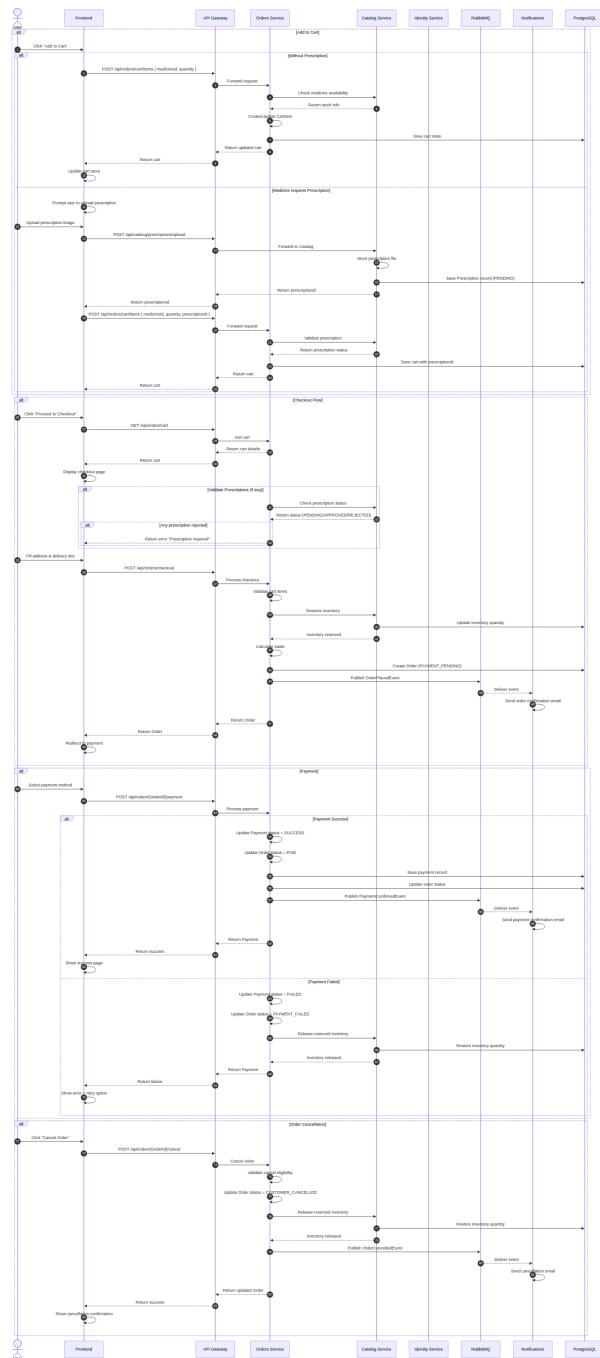


Figure 8 — Checkout & Payment Sequence Diagram